

App Name : Mano
To : NileAGI Technical Task Submission

Ipyana T Mwasajone



The Origin Story (Why "mano"?)

In many cultures, "Mano" translates directly to "Hand".

An AI assistant shouldn't feel like an unapproachable machine or a massive, cold cloud server cluster. It should feel like an extension of yourself—a helpful hand that guides you, creates with you, and organizes your digital environment

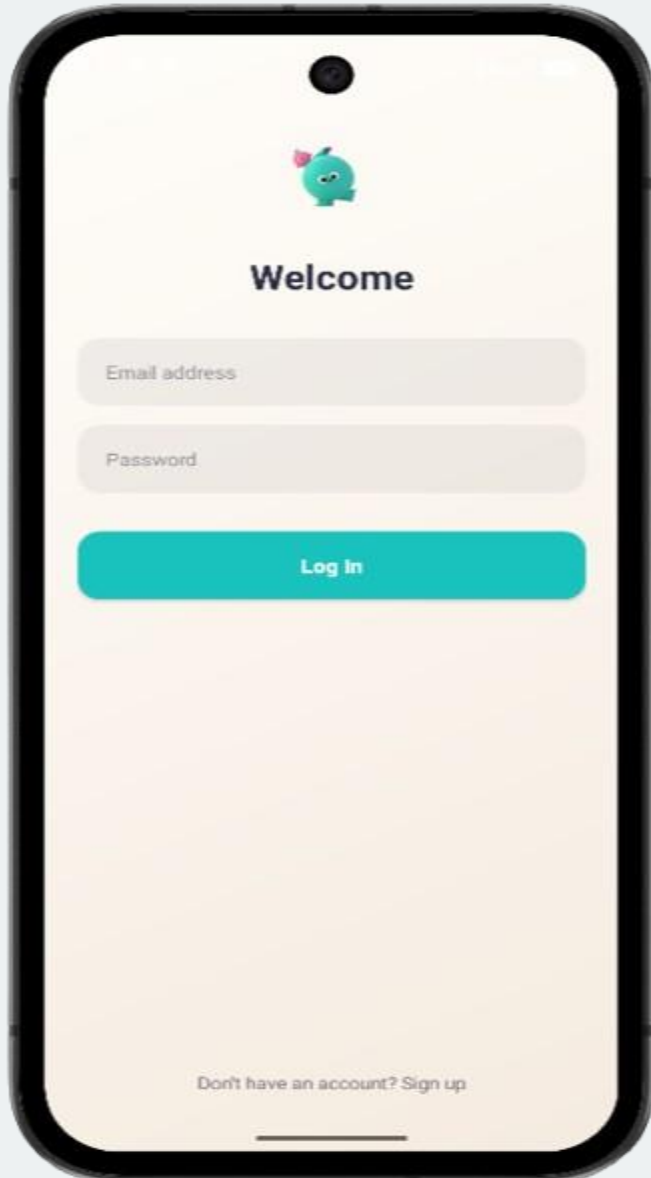
(mano) was intentionally selected to evoke minimalism, humility, and accessibility. It's a premium, high-end design pattern that sits quietly on a user's device, ready to assist without creating visual noise



Splash Screen

The initial launch gatekeeper and visual anchor of the application, designed to handle background system configurations before the user interacts with the app.

How long it loads: It stays visible for exactly 3 seconds—long enough to smoothly establish initial window bindings (EdgeToEdge layout processing) without making the application experience feel sluggish to the user.

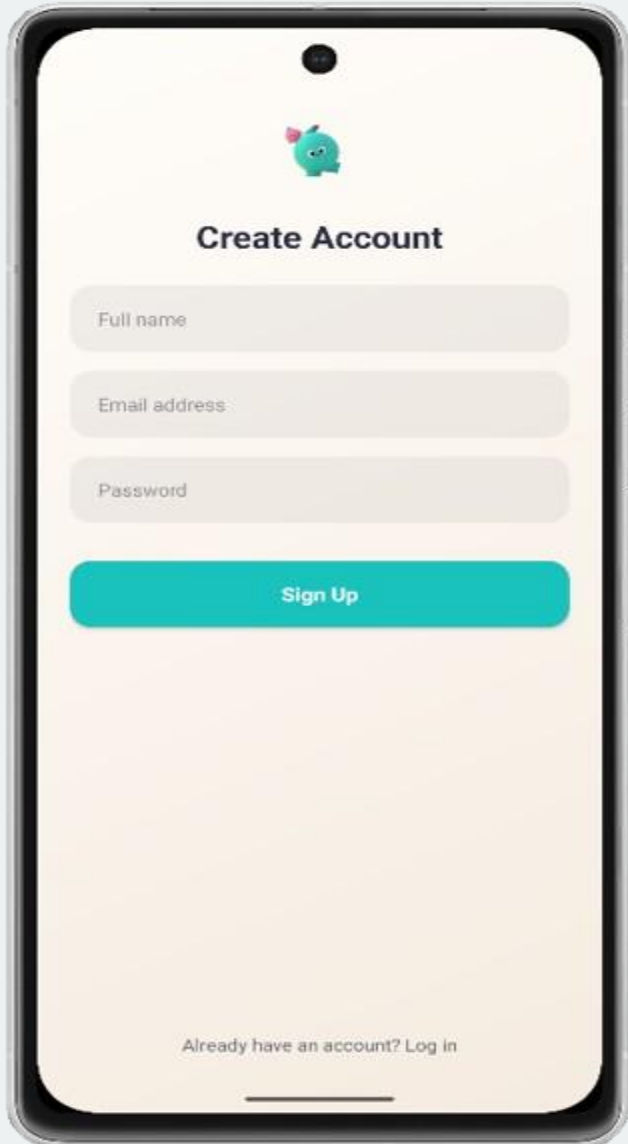


Login Screen

Sanitize: User hits "Log In" code instantly strips spaces and blocks empty inputs.

Verify: Locally queries the secure SQLite database (ManoUser.db) right on the device to match the user

.Launch: Confirms matching credentials and immediately flags an intent to switch screens, opening the main chat view without a lag.



Signup Screen

Capture & Clean: The user inputs their name, email, and password. The code instantly handles input sanitization, stripping trailing spaces and ensuring fields aren't blank.

Commit to Edge: The sanitized inputs are packaged and written securely into a new row inside the native, on-device SQLite database ledger (`ManoUser.db`).

Auto-Authenticate: Once the row successfully writes, the system immediately flags a valid session token state and routes the newly registered user straight into the core chat interface.



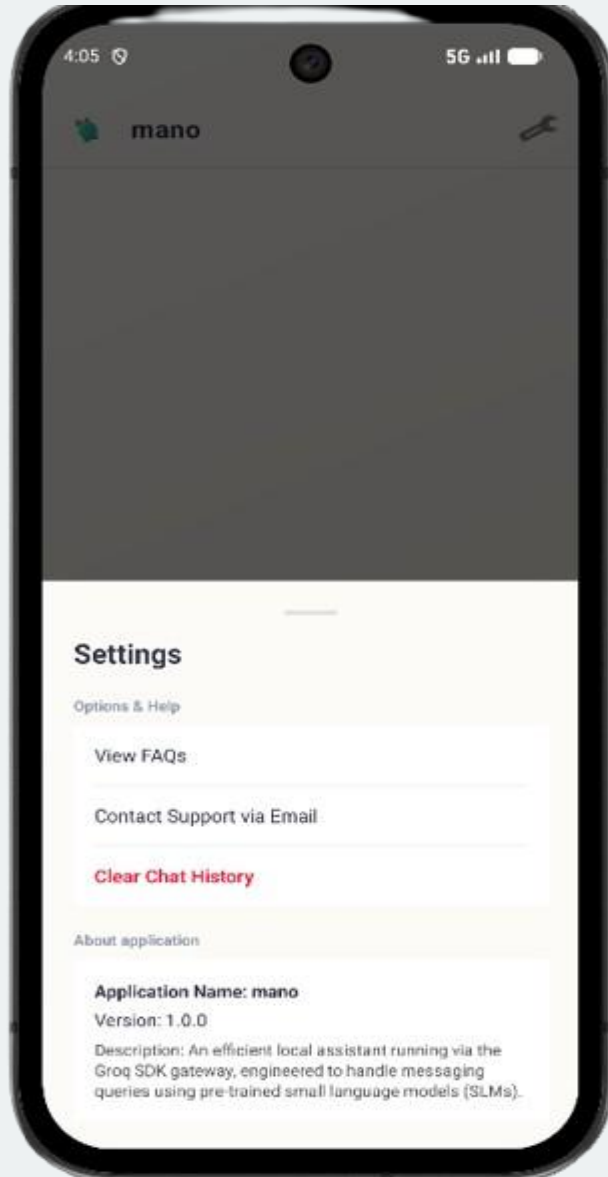
Dialog Screen

Send: User types a message; `RecyclerView` immediately paints their bubble on the right.

Wait: App injects a temporary loading state tracker, turning on an inline spinning loading indicator.

Fetch: `OkHttp` builds a JSON payload and safely fires an asynchronous background request to Groq Cloud (`11ama3-8b-8192`).

Render: The code catches the incoming message response string, kills the loading indicator, and instantly paints **mano's** reply bubble on the left.



Settings Screen

View FAQs: A clean navigational trigger routing users to helpful introductory documentation.

Contact Support via Email: An explicit intent wrapper setup that fires open the device's native mail client, auto-populating tech support parameters.

Clear Chat History: Highlighted intentionally in warning-state red typography, this button clears the active message database array list directly from the local SQLite layer.

Resources

Resources

Groq SDK Documentation: Sourced for optimizing payload serialization

Pinterest Creative Hub: Served as the core mood-boarding resource

Material Design 3 Guidelines (Google): Referenced for engineering the premium bottom sheet overlay system

OkHttp Official Cookbook: Referenced for setting up non-blocking asynchronous callback handling

Thank you

Ipyana T Mwasajone
0759182852